

CANONICAL FDR STRUCTURE

1. Document Information

Purpose:

Defines module metadata and ownership context.

Required Fields:

Field	Description
Module Name	Official flow/module name
Module ID	Flow numbering
Product	CrewAnywhere2.0
Document Type	Combined Lean FDR + Developer-Ready Technical FDR
Primary Actor	Main system actor
Related Modules	Connected modules
Status	Draft / Approved / Final

Format:

Structured table.

2. Overview

Purpose:

High-level explanation of the module.

Must include:

- business purpose
- operational purpose
- supported flows

- system responsibilities
- backend behaviors summary

Writing Style:

Narrative paragraph + bullet support.

3. Objectives

Purpose:

Defines operational goals.

Format:

Structured table.

Example Structure:

Objective
Allow secure onboarding
Prevent duplicate records
Validate user data

Must include:

- platform goals
 - operational outcomes
 - validation objectives
 - readiness goals
-

4. Actors

Purpose:

Defines all participating system actors/services.

Format:

Structured table.

Required Columns:

| Actor | Description |

Include:

- user roles
 - backend services
 - external systems
 - databases
 - notification systems
-

5. Preconditions

Purpose:

Conditions required before flow execution.

Format:

Structured table.

Examples:

- user not logged in
 - internet available
 - provider operational
 - permissions granted
-

6. Trigger

Purpose:

Defines exact event starting the flow.

Format:

Simple declarative statement.

Example:

“The process starts when:”

7. High-Level Architecture

Purpose:

Shows system interaction sequence.

Format:

Vertical architecture flow.

Example:

Frontend

↓

API Gateway

↓

Service Layer

↓

Database

↓

Notification Service

Must include:

- frontend
 - APIs
 - backend services
 - DB
 - external integrations
-

8. Main Process Flow

Purpose:

Core operational workflow.

Mandatory Formatting:

- Step-by-step structure
- Sequential numbering
- Explicit system actions
- Explicit user actions
- Explicit backend behaviors

Required Structure:

Step X — Flow Name

Contains:

- user action
- system action
- redirects
- validations
- backend processing
- state changes

Must include:

- conditional branching
 - loop handling
 - redirect logic
 - operational enforcement
-

9. Form Specification

Purpose:

Defines all form/input fields.

Format:

Structured table.

Required Columns:

| Field | Type | Required | Notes / Validation |

Must include:

- field typing
 - validation rules
 - optional/required logic
 - constraints
 - editable state
-

10. Terms & Conditions / Consent Layer

Purpose:

Legal/compliance handling.

Must include:

- frontend enforcement
 - backend storage logic
 - timestamp handling
 - mandatory consent behavior
-

11. Validation Rules

MANDATORY SECTION.

Must contain BOTH:

Frontend Validation

Format:

Validation table.

| Rule | Error |

Backend Validation

Must include:

- duplicate validation
 - token validation
 - sanitization
 - identity verification
 - rate limiting
 - dependency validation
 - business rule enforcement
-

12. Validation Failure Flow

Purpose:

Explains failed validation behavior.

Must include:

- inline errors
- retry logic
- prevention behavior
- edit loops
- resubmission logic

Structure:

Operational flow description.

13. Validation Success Flow

Purpose:

Defines successful execution behavior.

Must include:

- DB creation sequence
- system-generated actions
- status updates
- notifications
- audit logging
- token generation
- downstream flow initialization

Must be sequentially numbered.

14. Database Schema

MANDATORY DEVELOPER SECTION.

Must include:

14.X Table Name

Format:

| Field | Type |

Must include:

- PK/FK references
- timestamps
- ENUM references
- nullable behavior if applicable

Must include ALL related tables.

Example:

- primary entity table
 - auth table
 - verification table
 - mapping table
 - audit table
-

15. ENUM Definitions

Purpose:

Centralized ENUM architecture.

Format:

Section-based ENUM listing.

Example:

account_status

- ACTIVE
- SUSPENDED

Rules:

- use uppercase ENUMs
- no undocumented ENUMs

- lifecycle consistency mandatory
-

16. API Specifications

MANDATORY.

Each API must contain:

Endpoint Name

Endpoint

POST /api/v1/...

Responsibilities

- validation
- processing
- orchestration

Request Body

JSON example.

Success Response

JSON example.

Error Response

JSON example.

Must include:

- authentication behavior
 - permission requirements
 - lifecycle impacts
-

17. Lifecycle / Verification / Processing Flow

Purpose:

Defines operational lifecycle sequences.

Examples:

- email verification
- KYB review
- escrow lifecycle
- payout processing

Format:

Arrow-based lifecycle flow.

Example:

Created

→ Submitted

→ Pending Review

→ Approved

Must include:

- expiration rules
 - retry logic
 - dependency rules
 - automation behavior
-

18. Session Handling

Purpose:

Authentication/session restrictions.

Must include:

- verified/unverified behavior
- access restrictions
- token/session rules

- timeout behavior

Format:

Structured table.

19. Access Control Rules

Purpose:

Permission architecture.

Format:

Structured table.

| Condition | Access |

Must define:

- role-based access
 - readiness restrictions
 - state-based permissions
-

20. Security Requirements

MANDATORY.

Must include:

- hashing
- encryption
- CSRF protection
- sanitization
- HTTPS enforcement
- JWT/session handling
- rate limiting
- audit security

Format:

Structured requirement list.

21. Error Handling

MANDATORY.

Format:

Structured table.

| Scenario | System Response |

Must include:

- validation failures
 - DB failures
 - API failures
 - token failures
 - network failures
 - external provider failures
-

22. Notifications

Purpose:

System communication behaviors.

Format:

Structured table.

| Event | Notification |

Must include:

- email
 - push
 - system notifications
 - resend behaviors
-

23. Audit Logging

MANDATORY.

Must define:

- events logged
- security events
- lifecycle transitions
- retries
- failures
- admin actions

Format:

Bullet structure.

24. Business Rules

MANDATORY.

Purpose:

Defines non-technical operational rules.

Format:

Structured table.

| Rule | Description |

Must include:

- dependency rules
 - readiness logic
 - validation dependencies
 - lifecycle enforcement
 - uniqueness rules
-

25. Exception Flows

Purpose:

Defines abnormal operational flows.

Format:

Structured table.

| Scenario | System Response |

Must include:

- retries
 - redirect behavior
 - recovery logic
 - fallback handling
-

26. State Transition Logic

MANDATORY.

Purpose:

Defines status lifecycle architecture.

Format:

Arrow-based lifecycle.

Example:

DRAFT

→ OPEN
→ REVIEWING
→ FILLED

Rules:

- lifecycle consistency mandatory
 - must align with DB ENUMs
 - must align with business rules
-

27. UI Components

Purpose:

Defines frontend screens/components.

Format:

Structured list/table.

Must include:

- pages
 - modals
 - forms
 - validation states
 - dashboards
 - success/failure states
-

28. Postconditions

Purpose:

Defines final successful outcomes.

Format:

Structured table/list.

Must include:

- DB results
 - status outcomes
 - readiness state
 - downstream eligibility
-

29. Acceptance Criteria

MANDATORY QA SECTION.

Format:

| ID | Acceptance Criteria |

Rules:

- uniquely numbered
- testable
- implementation-verifiable

Must cover:

- happy paths
 - validation paths
 - security
 - lifecycle enforcement
 - DB creation
 - notifications
-

30. Next Connected Module

Purpose:

Defines downstream module continuation.

Format:

Declarative transition.

Must include:

- next operational module
 - downstream readiness
 - connected lifecycle
-

REQUIRED DOCUMENTATION STYLE RULES

Writing Style

Must use:

- infrastructure-style wording
- implementation language

- backend-operational terminology
- explicit system behavior wording

Avoid:

- vague product language
 - UX-only descriptions
 - simplified summaries
-

Mandatory Technical Depth

Every future FDR must include:

- ✓ SQL examples
 - ✓ Request/response payloads
 - ✓ Explicit backend logic
 - ✓ Validation branching
 - ✓ Lifecycle enforcement
 - ✓ ENUM definitions
 - ✓ State transition logic
 - ✓ Database schema typing
 - ✓ Operational system behavior
 - ✓ Access control rules
 - ✓ Security requirements
 - ✓ Audit logging
 - ✓ Exception handling
 - ✓ API responsibilities
-

Canonical Formatting Rules

Future FDRs must maintain:

Standard	Requirement
Section numbering	Fixed
Section order	Fixed

Naming style	Consistent
ENUM casing	UPPERCASE
API style	REST format
Database typing	Explicit
Validation structure	Frontend + Backend separated
Lifecycle format	Arrow flow
Acceptance criteria	AC-XX numbering

Official CrewAnywhere2.0 FDR Standard

Canonical baseline:

1.1 Business User Registration Process

All future modules should follow this exact documentation architecture:

- 1.2 Crew Registration
- 2.1 Company Profile Creation
- 2.2 Crew Profile Creation
 - 8. Event & Job Creation
 - 9. Job Application
 - 10. Talent Selection
 - 11. Payment Process
 - 12. Operational Execution
 - 13. Withdrawal Process

Consistent formatting across all modules is now critical for:

- developer onboarding
- backend implementation
- API consistency
- architecture governance
- scalability
- QA testing
- system integration
- enterprise documentation quality.